

SEGMENT 04

Sarvam-105B and Document AI

Reasoning, tool calling, and turning Indic paper into structured data

Chat Completions · reasoning_effort · tool use · Vision 1.5 · digitise & extract

Dr. Bhaveshkumar C. Dharmani · PhD (ICT), DA-IICT Gandhinagar · Founder, AI Vidhya4 Sarvam

<https://www.aividhya.in/> · <https://www.aividhya4sarvam.in/> · bhavesh@aividhya.in · <https://www.linkedin.com/in/bhaveshdharmani/>



Sarvam-105B

The ninety seconds that reframes the whole platform

Your existing code already works

OPENAI SDK — POINTED AT SARVAM. NOTHING ELSE CHANGES.

```
from openai import OpenAI

client = OpenAI(
    api_key="YOUR_SARVAM_KEY",
    base_url="https://api.sarvam.ai/v1",    # <- the only change
)

r = client.chat.completions.create(
    model="sarvam-105b",
    messages=[{"role": "user", "content": "GST का आर्थिक प्रभाव समझाइए"}],
```

LangChain, Vercel AI SDK, LlamaIndex, most agent frameworks

All work through the OpenAI-compatible endpoint. Your existing scaffolding is not wasted.

But the native SDK breaks convention in one place

With the sarvamai client it is `client.chat.completions(...)` — NOT `.create(...)`. This trips up every AI coding assistant that has not read the Agent Skills.

Ninety seconds, and the platform stops being 'another API to learn' and becomes 'a base URL'. Do this demo live, early, while attention is high.

The architecture, briefly

128

sparse experts in the
Mixture-of-Experts transformer

MLA

Multi-head Latent Attention —
compressed KV for long context

12T

pre-training tokens: code, math,
multilingual, web

128K

context window
(Sarvam-30B: 64K)

- Trained from scratch on IndiaAI Mission compute — not a fine-tune of someone else's base model. That distinction is what makes the sovereignty claim real rather than cosmetic.
- Released under Apache 2.0 alongside Sarvam-30B. You can download the weights and self-host. This is your insurance policy against pricing or terms changing.
- Powers Indus, Sarvam's own AI assistant — which is a useful signal about what the model is actually good at: long-horizon reasoning and agentic workflows.

The failure everyone hits in the first hour

WHAT PEOPLE WRITE

```
r = client.chat.completions(  
    model="sarvam-105b",  
    messages=[...],  
    max_tokens=200,  
)  
  
print(r.choices[0].message.content)  
# None
```

WHY

```
# reasoning is ON by default  
#   reasoning_effort="low"  
#  
# reasoning tokens count  
# AGAINST max_tokens.  
#  
# 200 tokens of budget,  
# all consumed thinking,  
# nothing left to say.
```

1 Fix A — give it room

- `max_tokens=2000`
- Keeps the reasoning, which is usually what you want
- Plan ceilings: Starter 4096 / Pro 16384 / Business 128000

2 Fix B — turn reasoning off

- `reasoning_effort=None`
- Faster and cheaper for simple extraction or classification
- Default temperature shifts 0.5 → 0.2 when reasoning is disabled

Every parameter that matters

Parameter	Range / default	When you touch it
model	sarvam-105b	The only current chat model. sarvam-m and sarvam-30b are deprecated — pass model= explicitly, always
reasoning_effort	low (default) / None	None for classification and extraction. Low for anything multi-step
temperature	0–2. 0.5 reasoning on, 0.2 off	0–0.3 for extraction, 0.7+ for generation
max_tokens	Starter 4096 / Pro 16K / Business 128K	Must leave room past reasoning tokens
top_p	0–1	Leave at 1 unless you know why
stream	true / false	true for any user-facing interface
seed	integer	Repeatable results. Essential for eval harnesses
stop	up to 4 sequences	Structured output boundaries
frequency_penalty	-2 to 2	Reduce repetition in long generation
presence_penalty	-2 to 2	Encourage topic movement
n	1–128	Multiple completions per request

Prompt caching is an architecture decision

₹29.28

input, per 1M tokens

₹10.98

CACHED input, per 1M tokens

62%

saving on the stable
part of your prompt

₹73.20

output, per 1M tokens

STRUCTURE YOUR PROMPTS SO THE STABLE PART IS CACHEABLE

```
messages = [  
  {"role": "system", "content": LONG_STABLE_INSTRUCTIONS + POLICY + EXAMPLES}, # cached  
  {"role": "user", "content": this_turn_only}, # not cached  
]  
# Put everything that does not change per request FIRST and keep it byte-identical.
```

₹

On a voice agent with an 8-turn conversation this saves roughly ₹0.36 a call. At 100,000 calls a month that is ₹36,000. From prompt ordering.

Tool calling — the foundation of everything agentic

PYTHON — a three-tool agent

```
tools = [{  
    "type": "function",  
    "function": {  
        "name": "get_emi_schedule",  
        "description": "Fetch the EMI schedule for a loan account",  
        "parameters": {  
            "type": "object",  
            "properties": {"account_id": {"type": "string", "description": "Loan account number"}},  
            "required": ["account_id"],  
        },  
    },  
}]
```

```
r = client.chat.completions(  
    model="sarvam-105b", messages=msgs, tools=tools, max_tokens=2000,
```



Why the benchmarks matter here

49.5 on BrowseComp and 68.3 on Tau2 — the highest among compared models. Those are the agentic scores, and they are the reason this model is worth building workflows on rather than just chatting with.



Document AI

Sarvam Vision 1.5 — the cheapest useful thing on the platform

Two endpoints, one job model

D Digitise — the whole document

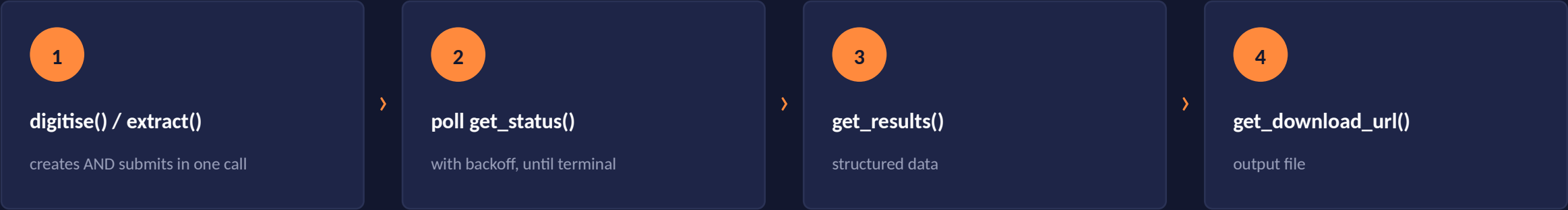
- Full-document OCR
- Preserves layout and reading order
- Tables parsed into HTML or Markdown
- Output: html or md, PLUS per-page JSON always
- Download is a ZIP: primary file + metadata/page_NNN.json + manifest.json
- For: archival, search, RAG ingestion, faithful conversion
- POST /doc-ai/v1/job/digitise

E Extract — just the fields you define

- Schema-based key-value extraction
- You supply a JSON schema (or a saved config_id)
- Output: JSON, CSV or XLSX
- For: KYC, invoices, forms, structured capture
- POST /doc-ai/v1/job/extract
- Provide exactly one of schema OR config_id — both or neither returns 400

★ **The rule: do not digitise a whole document to pull three fields. Write a schema and let Extract do it. Cheaper, faster, and far more reliable.**

The job lifecycle



State	Meaning	What you must do
pending	Created, queued	Keep polling
running	Being processed	Watch usage.pages_processed
completed	All pages succeeded	Fetch results
partially_completed	Some pages succeeded, some failed	TERMINAL. Handle it — most people don't, and silently lose pages
failed	All pages failed or job-level error	Log and retry
rejected	Rejected before processing	Validation or entitlement problem



partially_completed is a terminal state, not a transient one. If your code only checks for 'completed' you will lose pages and never know.

Writing a schema that actually works

PYTHON

```
schema = {
    "type": "object",
    "properties": {
        "policy_number": {
            "type": "string",
            "description": "Insurance policy number,
                top-right, format ABC-1234"},
        "insured_name": {
            "type": "string",
            "description": "Name of the insured person"},
        "sum_insured": {
            "type": "number",
            "description": "Total sum insured, in INR"},
    },
}

job = client.doc_ai.extract(
    file=[("policy.pdf", f, "application/pdf")],
    schema=json.dumps(schema), # STRING not dict
    language="en-IN",
    output_format="json",
)
```

1

The description IS the prompt

"Insurance policy number, top-right, format ABC-1234" beats "policy number" by a wide margin. Be specific about location and format.

2

Schema rules

Root must be type object with non-empty properties. Every field needs a type and a non-empty description. Types: string, number, integer, boolean, object, array. Optional enum. Max nesting depth 4.

3

Pass it as a JSON string

It is sent as a multipart form field. `json.dumps()` in Python, `JSON.stringify()` in JS. A dict raises `AttributeError: 'dict' object has no attribute 'read'`.

The naming gotchas — all six

You write	What happens	Correct
language_code="hi-IN"	Silently ignored. No error	language="hi-IN"
output_format="markdown"	400 Bad Request	output_format="md"
schema={...} (a dict)	AttributeError: 'dict' has no attribute 'read'	schema=json.dumps({...})
file=open(...)	Type error	file=[(name, handle, mimetype)] — an array
getStatus({job_id}) in JS	undefined fields	getStatus(job_id) — positional, wire names not camelCase
11-page PDF	400 invalid_request_error	Chunk to 10 pages maximum



Document AI uses a different parameter vocabulary from STT, translate and LID. That inconsistency is exactly why the Agent Skills exist.

The limits that shape your architecture

10

pages per job
(PDF or ZIP)

200 MB

maximum file size

10/min

requests per minute —
ON EVERY PLAN

₹0.50

per page
(was ₹1.50 until May 2026)

- The 10 req/min cap is not a tier limit — Starter, Pro and Business all get it. Your architecture needs a queue, not a bigger plan.
- Thirty people in this room submitting simultaneously will hit it instantly. That is why we are running this lab in waves — and experiencing the constraint is the lesson.
- At ₹0.50 a page, digitising 100,000 pages of legacy Indic records costs ₹50,000. A BPO charges ₹3–8 a page. Sit with that gap for a moment.

Error handling you will actually need

Status	Meaning	What to do
400	Invalid request or schema, or over 10 pages	Fix the payload. Chunk the document
402 / 403	Billing or entitlement	Check credits. Not retryable
404	Job not found	Check the job_id
409	Results requested before terminal state	Poll get_status first
413	File too large	Split it
422	Invalid format or corrupted file	Validate before submitting
429	Rate or admission limit	Exponential backoff with jitter
503	Billing unavailable	Retry with backoff



Build the retry logic before you need it. A pipeline that dies on one corrupted file in a batch of ten thousand is not a pipeline.

The man who read the whole newspaper for the cricket score

Every morning he read all forty pages, front to back, to find one number.

It took an hour. It was thorough. It was correct.

His neighbour turned to page 14, read the score, and made tea.

Both men got the same number. One of them got an hour back.



E

Digitise reads the whole newspaper. Extract turns to page 14. If you know which three fields you need, define a schema — do not OCR forty pages and regex your way through the result.

What this segment cost you

Service	Input	Cached input	Output
Sarvam-105B	₹29.28 / 1M	₹10.98 / 1M	₹73.20 / 1M
Sarvam-105B Chat	₹29.28 / 1M	₹10.98 / 1M	₹73.20 / 1M
Gemma-4 31B	₹36.60 / 1M	₹13.73 / 1M	₹91.50 / 1M
GLM 5.2	₹128.10 / 1M	₹23.79 / 1M	₹402.60 / 1M
Document Digitisation	₹0.50 per page	—	max 10 pages/job



Notice something

The Indic-native flagship is also the cheapest of the three chat models on the platform. GLM 5.2 costs 4.4x more on input and 5.5x more on output. That is not an accident — it is a defensible cost advantage for anyone building India-first.



Your Lab 4 agent: about ₹0.70 for ten conversations. Lab 5 document pipeline: about ₹10 for twenty pages. Check your dashboard.

“

The reasoning model is 9% of your bill.

The part everyone optimises is the part that barely matters.

— A preview of the money segment. Next: Indus.